Chapter - 6

Constructors and Destructors

Key Concepts

- Constructing objects
- Constructors
- Constructor overloading
- Default argument constructor
- Copy constructor
- Constructing matrix objects
- Automatic initialization
- Parameterized constructors
- Default constructors
- Dynamic initialization
- Dynamic constructor
- Destructors

6.1 Introduction

- Member functions such as putdata() and setdata() provide initial values to private member variables.
- Example:

  A.input() assigns values to data members.
- Example:

  x.getdata(100, 299.95) passes initial values as arguments to set data in objects.
- These functions cannot initialize member variables at object creation.

Philosophy of C++ Initialization

- C++ allows user-defined data types (classes) similar to built-in types.
- A class type variable should be initialized when declared, like built-in types.

  Example: int m = 20; float x = 5.75;
- Destructor automatically deletes objects after scope ends.
- Special member functions:

  Constructor: Initializes objects at creation.

  Destructor: Destroys objects when no longer required.

6.2 Constructors

- A constructor is a special member function used to initialize objects of its class.
- It has the same name as the class.
- Automatically invoked when an object of the class is created.
- Called constructor because it constructs the values of data members.

Example:

```
class integer {
  int m, n;
public:
  integer(void);   // constructor declared
};

integer::integer(void) {   // constructor defined
  m = 0; n = 0;
}
```

Default Constructor

- When a class contains a constructor, any object created will be initialized automatically.

  Example:

  integer int1; // object int1 created

- This initializes data members (m, n) to zero without explicit function call.
- A constructor with no parameters is called the default constructor.
- Default constructor for class A is A::A().
- If no constructor is defined, the compiler supplies a default constructor.
- Example:

  A a; // invokes default constructor to create object a.

Characteristics of Constructors

- Declared in the public section of a class.
- Invoked automatically when objects are created.
- Do not have return types, not even void.
- Cannot be inherited (though derived class can call base class constructor).
- Can have default arguments.
- Constructors cannot be virtual.
- Cannot refer to their addresses.
- Object with constructor/destructor cannot be a union member.
- Implicitly call operators new and delete for memory allocation.

Note: Declaring a constructor makes object initialization mandatory.

6.3 Parameterized Constructors

- A constructor initializes data members of objects.
- Default constructor initializes all data members to zero.
- In practice, objects may need different initial values.
- C++ allows passing arguments to constructors.
- This enables initialization of objects with different values.
- Such constructors are called parameterized constructors.

Example

```
class integer
{
    int m, n;
    public:
        integer(int x, int y);  // parameterized constructor
};

integer::integer(int x, int y)
{
    m = x;
    n = y;
}
```

Object Creation with Parameterized Constructors

- Normal declaration like:

```
integer int1;
```

❌ **Not valid** (no default constructor defined).

- We must pass **initial values as arguments** when declaring objects.

Explicit Call

```
integer int1 = integer(0, 100);   // explicit call
```

- Creates object `int1` and passes **0 and 100** to the constructor.
- Longer form of initialization.

Implicit Call

```
integer int1(0, 100);   // implicit call
```

- Creates object `int1` with **0 and 100** directly.
- Called **shorthand method**.

Continue…

- When using **parameterized constructors**:
  - Must always provide arguments.
  - No automatic default initialization unless a **default constructor** is defined.

Continue…

The constructor functions can also be defined as **inline** functions. Example:

```
class integer
{
      int m, n;
  public:
      integer(int x, int y)    // Inline constructor
      {
            m = x; y = n;
      }
      .....
      .....
};
```

Continue…

The parameters of a constructor can be of any type except that of the class to which it belongs. For example,

```
class A
{
        .....
        .....
    public:
        A(A);
};
```

is illegal.

Continue…

However, a constructor can accept a *reference* to its own class as a parameter. Thus, the statement

```
Class A
{
      .....
      .....
  public:
      A(A&);
};
```

is valid. In such cases, the constructor is called the *copy constructor*.

6.4 Multiple Constructors in a Class

- Constructors are special functions used for object initialization.
- A class can have multiple constructors with different argument lists.
- This is called Constructor Overloading.

Types of Constructors

1. Constructor with no arguments.
2. Constructor with two integer arguments.
3. Constructor with one object as argument (Copy Constructor).

Example: Class integer

```
class integer {
   int m, n;
public:
   integer() {m=0; n=0;}       // Constructor 1
   integer(int a, int b) {m=a; n=b;}   // Constructor 2
   integer(integer &i) {m=i.m; n=i.n;} // Constructor 3
};
```

Usage Examples

- integer I1; // Calls constructor 1 (m=0, n=0)
- integer I2(20,40); // Calls constructor 2 (m=20, n=40)
- integer I3(I2); // Calls constructor 3 (copy constructor)

Key Points

- Constructor overloading allows multiple constructors in the same class.
- The compiler chooses the constructor based on the arguments.
- Copy constructor is used to create a new object as a copy of an existing one.

6.5 Constructors with Default Arguments

- Constructors can have default arguments.
- Default values are used when arguments are not provided.
- Helps simplify object initialization.

Example Declaration

```
complex(float real, float imag = 0);
```

- The default value of 'imag' is 0.
- If only 'real' is provided, 'imag' is automatically set to 0.

Example Usage

```
complex C(5.0);
→ real = 5.0, imag = 0.0 (default)

complex C(2.0, 3.0);
→ real = 2.0, imag = 3.0 (explicit values)
```

Key Rule

- Default arguments must be trailing (from right to left).
- Missing arguments are replaced by their default values.

Default Constructor vs Default Argument Constructor

- Default Constructor: A::A()
- Default Argument Constructor: A::A(int = 0)

Ambiguity arises when both are defined in the same class:

```
A a; // Compiler confusion: call A::A() or A::A(int=0)?
```

6.6 Dynamic Initialization of Objects

- Class objects can be initialized dynamically.
- Initial value of an object may be provided during run time.
- Advantage: Provides various initialization formats using overloaded constructors.
- Flexibility: Different formats of data can be used at run time depending on the situation.

Example Context

- Consider long term deposit schemes in commercial banks.
- Banks provide different interest rates for different schemes.
- Rates vary for different periods of investment.
- Program 6.3 illustrates the use of class variables for account details.

Dynamic Initialization in Practice

- Class variables can hold account details.
- Constructors dynamically initialize variables during run time.
- Flexibility for different schemes and investment durations.
- Demonstrates practical use of object-oriented programming in banking applications.

6.7 Copy Constructor

- Copy constructor is used to declare and initialize an object from another object.
- Syntax: integer(integer &i);
- It is one of the overloaded constructors.
- Provides a way to duplicate objects during creation.

Example of Copy Constructor

- Example statement:

  integer I2(I1);

- This defines the object I2 and initializes it to the values of I1.
- Another form:

  integer I2 = I1;

Copy Initialization

- The process of initializing through a copy constructor is called copy initialization.
- Example:

  I2 = I1;

- Ensures new object is initialized with values of an existing object.

Assignment vs Copy Constructor

- Statement I2 = I1 does not invoke the copy constructor.
- Instead, it assigns values of I1 to I2, member-by-member.
- This is handled by the overloaded assignment operator (=).
- Copy constructor is invoked only when a new object is created from an existing object.

Copy Constructor Argument

- A copy constructor takes a reference to an object of the same class.
- Syntax example:

  ```
  ClassName(const ClassName &obj);
  ```

- Ensures that an object can be initialized from another object of the same class.

Example Preview

- Program demonstrates constructing and using a copy constructor.
- Shows how objects are copied safely.
- Important for managing resources and avoiding shallow copies.

6.8 Dynamic Constructors

- Constructors can be used to allocate memory while creating objects.
- Ensures the system allocates the right amount of memory for each object.
- Useful when objects are not of the same size, saving memory.

Dynamic Construction of Objects

- Allocation of memory to objects at the time of their construction.
- Known as dynamic construction of objects.
- Memory is allocated using the 'new' operator.

6.9 Constructing Two Dimensional Arrays

- Only program

6.10 const Objects

- We may create and use constant objects using the **const** keyword before object declaration.
- Eg:

```
const matrix X(m,n); // object X is constant
```

- Any attempt made to modify the values of m and n will generate compile-time error.
- A **const** object can only call **const** member functions
- Whenever const objects try to invoke non-const member functions, the compiler generates an error

Properties of Const Objects

- Attempting to modify values of a const object generates compile-time error.
- A const object can call only const member functions.
- A const member is a function where 'const' appears after the function's signature.

Restrictions

- Const objects cannot modify data members.
- Const objects cannot invoke non-const member functions.
- Compiler generates errors when non-const functions are called.

6.11 Destructors

- A destructor is used to destroy objects created by a constructor.
- Destructor is a member function with the same name as the class but preceded by a tilde (~).
- Example: ~integer() { }

Properties of Destructors

- Destructor never takes arguments nor returns values.
- Automatically invoked by the compiler when the program exits or objects go out of scope.
- Good practice to declare destructors to release memory for future use.

Memory Management with Destructors

- When 'new' is used to allocate memory in constructors, 'delete' should be used in destructors.
- Example (Matrix class):

```
matrix :: ~matrix() {
    for(int i=0; i<d1; i++)
        delete p[i];
    delete p;
}
```

Key Notes

- Destructors are invoked implicitly.
- Important for cleaning up dynamically allocated memory.
- Ensures efficient memory management by freeing unused resources.